

# 工作交接0705

---

## 1.项目背景与进度

### 1.1 项目背景

这个项目的总体目标是，用大规模的数据集，去预训练一个我们自己的基模VLA。

目前此项目停留在第一阶段：目的是跑通基于大量数据集的预训练，并验证加入Ego数据对VLA预训练的有效性。

其中，在此阶段，VLA的架构直接参考pi05。

具体而言，为了完成第一阶段，需要做以下工作：

- 收集大规模的真机数据集和ego数据集（1500h 真机 + 1500h ego）
- 跑通基于这么多数据集的预训练（基于openpi代码的pi05 SFT）

### 1.2 数据集

在此介绍数据集的处理。

我们下载了很多数据集（RoboCOIN, Agibot, EgoVerse等开源数据集），数据集的具体构成见【预训练数据集总览\_v1】

这些数据集，会被转换成RLDS格式。每个RLDS仓库具有统一的格式Schema。基于这个前提，最终我们产出了40余个RLDS仓库

然后，预训练代码会读取这40多个仓库的数据，进行预训练

### 1.3 预训练代码

预训练的代码库，是徐乐同学构建的，路径在乌兰察布区H800机器的/mnt/workspace/xule/pi07\_reproduction

此代码库就是基于Openpi这个开源仓库改的

具体而言，在openpi的基础上，完成了以下功能：

- 对预训练数据集的接入处理：
  - openpi原始仓库是基于lerobot数据格式训练的，不适用于预训练。因此，书写了基于RLDS数据的接入适配层
- 将数据的action type这个meta信息，加入到提示词中
  - 即，VLA的输入除了语言指令外，现在还会有这个数据集的动作类型这样一个字符串。

目前此代码库已经可以完成基于3000h数据的预训练。

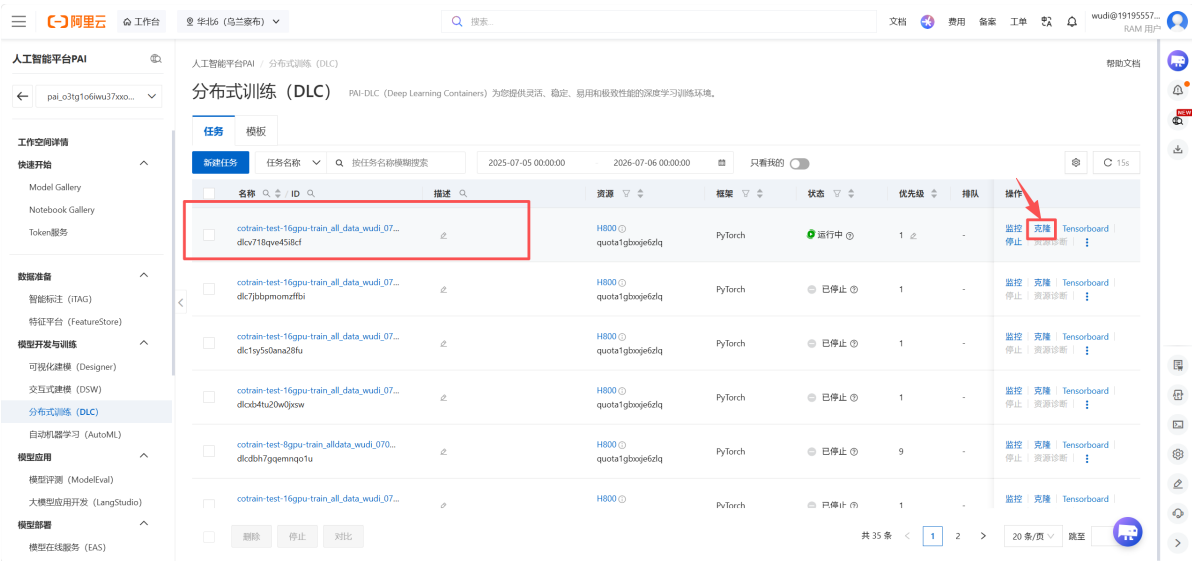
关于上面每个功能的具体实现代码细节，建议可以直接问大模型，让大模型读/mnt/workspace/xule/pi07\_reproduction 代码

## 2.如何复现现在的进度

当前已经可以完成的事情是，用H800 16卡去跑这样一个3000h数据的预训练

由于是16卡训练，所以需要在阿里云的【分布式训练】中跑，而无法在开发机中去跑

具体跑的方式，可以直接复制我现在的DLC训练，克隆然后运行即可：



为了更清晰的说明，这里也提供一个8卡训练的测试脚本，以及16卡DLC训练实际在使用的脚本：

### all data 8卡训练

```
1 bash -c 'cd /mnt/data/xule/pi07_reproduction
2 ln -s /mnt/data/cache/huggingface /root/.cache/huggingface
3 ln -s /mnt/data/cache/openpi /root/.cache/openpi
4 ln -s /mnt/data/cache/uv /root/.cache/uv
5 export
6 PYTHONPATH=/mnt/data/xule/pi07_reproduction/src:/mnt/data/xule/pi07_reproduc
7 tion/packages/openpi-client/src:$PYTHONPATH
8 export
9 WANDB_API_KEY=wandb_v1_8VnE2CPAsy1AI0uG8OcXwNkUQhm_US7Bouh3A2sR6BfLPPQ4eHi i3
10 tBHwcZIF8joITu3B8y2Lo2wn
11 export XLA_PYTHON_CLIENT_MEM_FRACTION=0.9
12
13 .venv/bin/python scripts/train_cotrain.py cotrain_full_all_full_norm \
14 --exp_name=cotrain_full_all_full_norm_8gpus_0702 \
15 --fsdp_devices 8 \
16 --batch-size 256 \
17 --num-train-steps 36000 \
18 --data-num-parallel-reads 1 \
19 --data-num-parallel-calls 2 \
20 --overwrite \
21 2>&1 | tee
22 /mnt/data/xule/pi07_reproduction/dlc_full_all_full_norm_8gpus_0703test_${RAN
23 K:-0}.log'
```

8卡的训练可以在开发机中跑

开发机的环境，可以直接启动之前徐乐的实例，然后直接跑上面的命令，就可以启动训练：

交互式建模 (DSW) 为AI开发者量身定制的云端机器学习交互式开发IDE，随时随地开启Notebook快速读取数据、开发算法、训练及部署模型。

开发机实例 Notebook BETA

新建实例 实例名称 按实例名称模糊搜索 创建起始日期 创建结束日期 只看我的 5s

| 实例名称 / ID                                    | 资源                       | 可见范围   | 状态  | 关机策略 | 优先级 | 排队 | 指定节点       | 操作                                    |
|----------------------------------------------|--------------------------|--------|-----|------|-----|----|------------|---------------------------------------|
| online-exp2<br>dsw-04cq28bzgkgyg35ov6        | H800<br>quota1gboxje6zql | 仅创建者可见 | 已停止 | -    | 9   | -  | 未开启        | 打开   启动   保存镜像   变更配置   定时关机设置        |
| online_exp<br>dsw-tu6twn62h67lrx5k           | H800<br>quota1gboxje6zql | 仅创建者可见 | 已停止 | -    | 9   | -  | 未开启        | 打开   启动   保存镜像   变更配置   定时关机设置        |
| Zdc_cpu<br>dsw-vv94k7wjz8gqpfqv              | H800<br>quota1gboxje6zql | 仅创建者可见 | 运行中 | -    | 9   | -  | 未开启        | 打开   停止   保存镜像   变更配置   定时关机设置        |
| sunqing_cpu_dev<br>dsw-ajja3a21ge4lkdf4      | H800<br>quota1gboxje6zql | 仅创建者可见 | 已停止 | -    | 1   | -  | 未开启        | 打开   启动   保存镜像   变更配置   定时关机设置        |
| ymq-egocentric<br>dsw-eq2x8wg7f3k3qrobed     | H800<br>quota1gboxje6zql | 仅创建者可见 | 已停止 | -    | 9   | -  | 已指定<br>已排除 | 打开   启动   保存镜像   变更配置   定时关机设置        |
| le_cpu<br>dsw-g0eshcg00478adw6m              | H800<br>quota1gboxje6zql | 仅创建者可见 | 运行中 | -    | 9   | -  | 未开启        | 打开   停止   保存镜像   变更配置   定时关机设置        |
| le-openpi-test2_8gpu<br>dsw-f6l9goafq8cg3wci | H800<br>quota1gboxje6zql | 仅创建者可见 | 已停止 | -    | 1   | -  | 未开启        | 打开   <b>启动</b>   保存镜像   变更配置   定时关机设置 |
| cpu_process_2<br>dsw-17e3cme3cd7ntuj8q       | H800<br>quota1gboxje6zql | 仅创建者可见 | 已停止 | -    | 9   | -  | 未开启        | 打开   启动   保存镜像   变更配置   定时关机设置        |

批量删除 批量停止 共 20 条

## all data 16卡训练

```

1 bash -c 'cd /mnt/data/xule/pi07_reproduction
2 ln -s /mnt/data/cache/huggingface /root/.cache/huggingface
3 ln -s /mnt/data/cache/openpi /root/.cache/openpi
4 ln -s /mnt/data/cache/uv /root/.cache/uv
5 export
6 PYTHONPATH=/mnt/data/xule/pi07_reproduction/src:/mnt/data/xule/pi07_reproduc
7 tion/packages/openpi-client/src:$PYTHONPATH
8 export JAX_COORDINATOR_ADDRESS=${MASTER_ADDR}:29500
9 export
10 WANDB_API_KEY=wandb_v1_8VnE2CPAsy1AI0uG8OcXwNkUQhm_US7Bouh3A2sR6BfLPPQ4eHi i3
11 tBHwCZIF8joITu3B8y2Lo2wn
12 export XLA_PYTHON_CLIENT_MEM_FRACTION=0.9
13
14 .venv/bin/python scripts/train_cotrain.py cotrain_full_all_full_norm \
15 --exp_name=cotrain_full_all_full_norm_16gpus_0703_test \
16 --fsdp_devices 8 \
17 --batch-size 512 \
18 --num-train-steps 3000000 \
19 --data-num-parallel-reads 1 \
20 --data-num-parallel-calls 2 \
21 --overwrite \
22 2>&1 | tee
23 /mnt/data/xule/pi07_reproduction/dlc_run_16gpu_0703_${RANK:-0}.log'
```

16卡训练只能通过DLC跑。直接复制我上面的那个DLC训练即可

注意：

- WANDB\_API\_KEY是我自己wandb项目的key，不过我已经把这个项目改成public了
- cotrain\_full\_all\_full\_norm 是数据集配置的名称。这个名称的配置会使用所有的3000h RLDS数据集
- exp\_name 和 写入log名称酌情修改

- num-train-steps 决定了训练的总步数。即，训练时间本质上只和training step有关，training step是由我们设置的。目前估计了一下，按每个数据集样本都训一次，共需3000000步。
- 除此之外的一些训练超参数的设置（例如学习率等），需要在mnt/workspace/xule/pi07\_reproduction/src/openpi/cotrain/config.py 中进行修改

## 3.目前遇到的问题和下一步的目标

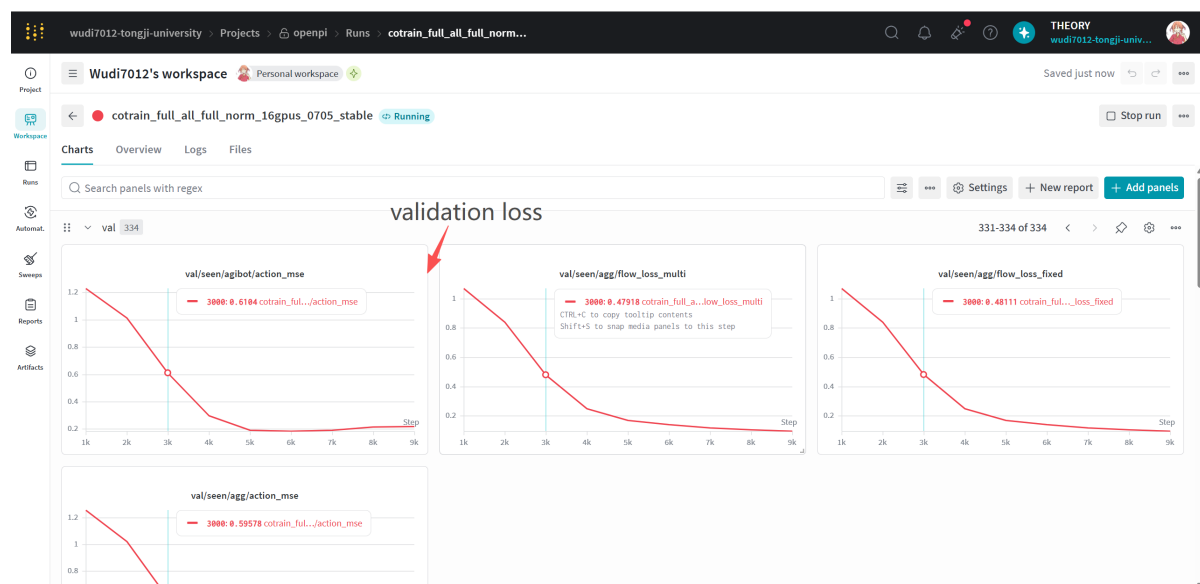
### 3.1 当前的现状

目前，我已经尝试跑了两个训练，wandb的网址分别是：

- <https://wandb.ai/wudi7012-tongji-university/openpi/runs/35om2fsv>
- <https://wandb.ai/wudi7012-tongji-university/openpi/runs/poylsk06>

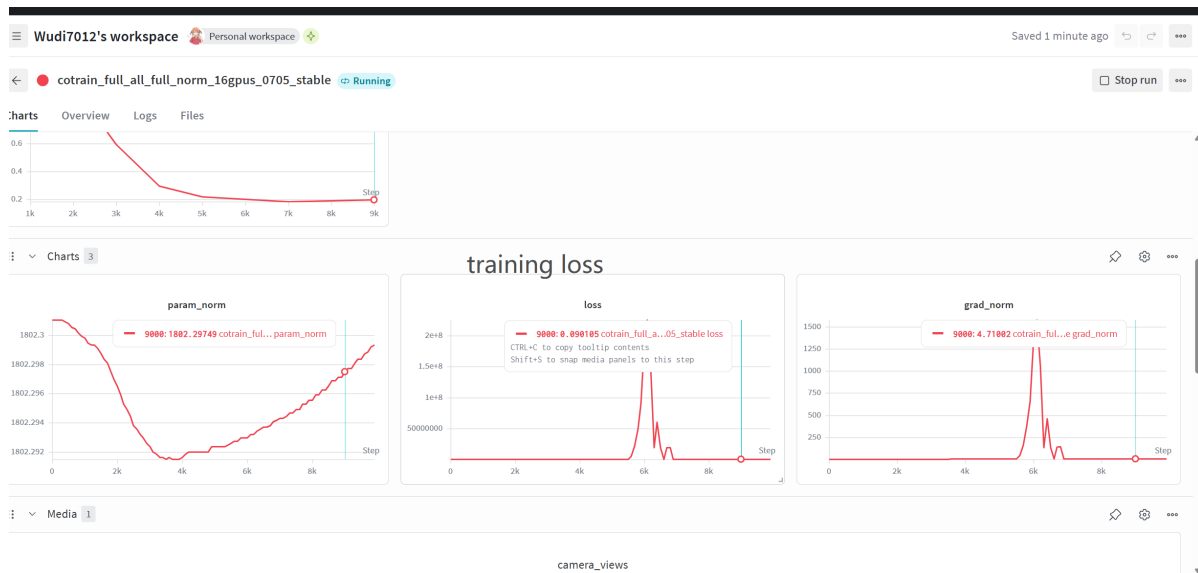
打开wandb页面，可以看到每个RLDS数据集的各自validation loss，以及训练的loss

validation是每隔1000步进行验证，我们在构建数据集时，已经划分了训练集和验证集。其中验证集由seen\_val和unseen\_val两部分组成，seen就是训练集中拿到，unseen就是训练集中没有的



下面有training loss

目前最大的问题是，训练loss不稳定，如下图所示，loss在6000step有一个突起



## 3.2 具体的问题

如wandb所示，现在最大的问题就是预训练的曲线loss会有突起，并不是一个稳定下降的状态

注意到我跑了两个wandb训练，当时跑完第一个wandb，也有这个问题，排查并改进了如下的问题：

1. robocoin\_unitree\_g1\_dex3\_s28\_a28和robomind\_tienkung\_sim\_s38\_a38这两个数据集loss异常  
很有可能这两个数据有问题，分布异常导致训练崩了
- 2.修改一下超参数，加强训练稳定性（grad norm限制，学习率调整）

修改完上面的两个细节后，跑了第二个wandb（就是上面图中的wandb），这个训练validation loss非常正常，但训练loss依然会有一个突起

**总结：现在主要的问题，就是这样一个大规模预训练跑起来loss异常，看起来像是训崩了**

## 3.3 后续改进目标

为了解决这个问题，陈老师提出了一个改进方向

陈老师提出的问题是，现在的动作空间不是固定第n个维度有某个明确的物理意义

即，我们有40个RLDS数据集，每个RLDS数据集有着自己的schema（即，有的是8维action，有的是16维action，最多的灵巧手有50多个action维度）

但现在的做法是，直接将这RLDS数据集混在一起训练，不做任何处理，将不同数据集的action直接丢到最终的一个64维动作空间中。

而陈老师希望的是，定义好64维度（或者更多维度）每个维度的物理意义，然后根据物理意义，去对不同数据集动作的不同维度进行分别的映射，空缺的部分用zero-padding处理。

**总而言之，下一步整体的目标就是要把大规模预训练成功训起来，解决现在看起来好像训崩了的问题。**

**其中，陈老师提的一个改进点，就是固定动作空间维度物理意义这个点**